Requerimientos del Proyecto — MediTurnos

Oct 17, 2026 · @Paula

Resumen y objetivo

MediTurnos es una app móvil (Android + iOS) para que pacientes soliciten, consulten y cancelen turnos médicos, con disponibilidad del equipo (médicos, secretarios, administrador) y funcionamiento offline-first: el paciente puede ver y pedir turnos aunque no tenga conexión, y la app sincroniza automáticamente cuando vuelve la señal.

Este documento junta todo lo que ya definimos en el informe (Partes I a IV) en una lista concreta de qué hay que construir, para repartirnos el trabajo entre front-end (pantallas) y back-end (API + base de datos).

Stack tecnológico acordado

Ya quedó decidido en Partes II y III, no se vuelve a discutir: se arranca directo con esto.

Capa	Tecnología	Nota
App	Flutter + Dart	Multiplataforma, un solo código para Android e iOS
IDE	Android Studio	Con plugin Flutter/Dart
Arquitectura	MVVM + Clean Architecture	Domain / Data / Presentation
Almacenamiento local	SQLite	Offline-first, cola de sincronización
Backend / API	Azure App Service	API REST que conecta la app con MySQL
Base de datos	MySQL (Azure Database for MySQL)	Ya scrapeada (ver script SQL): especialidades, especialistas, pacientes, secretarios, turnos, usuarios, historialclinico, notificaciones

Capa	Tecnología	Nota
Autenticación	JWT propio, emitido por la API	Token incluye el rol del usuario (paciente, especialista, secretario, admin)
Notificaciones push	Firebase Cloud Messaging (FCM)	Recordatorios de turnos
Otros servicios	WhatsApp Cloud API, SendGrid, Azure Notification Hubs	Canales complementarios de aviso

Modelo de datos

Basado en el script SQL ya corrido en MySQL (`mediturnos.sql`). 8 tablas, todas vacías por ahora (listas para cargar datos de prueba).

Tabla	Campos clave	Se relaciona con
`usuarios`	id_usuario, usuario, contrasena_hash, rol (paciente/secretario/especialista/admin)	Base para login (JWT)
`pacientes`	id_paciente, nombre, apellido, domicilio, teléfono, obraSocial, usuarioID	1 a 1 con `usuarios`
`secretarios`	id_secretario, nombre, apellido, DNI, usuario	1 a 1 con `usuarios`
`especialidades`	id_especialidad, nombre	Padre de `especialistas`
`especialistas`	id_especialista, nombre, apellido, DNI, especialidad	Pertenece a una `especialidad`
`turnos`	id_turno, fecha, hora, paciente, especialista, estado (activo/cancelado/reprogramado)	Une `pacientes` + `especialistas`
`historialclinico`	id_historial, pacienteDNI, especialistaID, fecha, diagnostico, tratamiento	Une `pacientes` + `especialistas`
`notificaciones`	id_notificacion, mensaje, tipo (recordatorio/aviso), usuario, turno	Une `usuarios` + `turnos`

Pendiente a resolver en equipo: la tabla `especialistas` no tiene una columna que la vincule a `usuarios` (a diferencia de `pacientes` y `secretarios`, que sí tienen `usuarioID` / `usuario`). Como el sistema de autenticación con JWT necesita que un médico también pueda loguearse, hay que agregar esa columna (`usuarioID`) antes de empezar el backend de login.

Requerimientos Funcionales

Mismo contenido de pantallas y API de antes, reescrito en el formato con el que venimos trabajando (Entrada / Proceso / Salida), numerado y agrupado en 4 bloques para repartir por persona.

Grupo 1 — Cuentas y acceso

1. El sistema debe permitir dar de alta un paciente en la base de datos.

Entrada: DNI, nombre, apellido, fecha de nacimiento, domicilio, teléfono, obra social (opcional).

Proceso: El sistema verifica que no exista ya un registro con ese DNI en la tabla pacientes (id_paciente). Si no existe, inserta el nuevo registro con usuarioID en nulo (el paciente queda dado de alta pero todavía sin acceso con usuario y contraseña).

Salida: Mensaje de alta exitosa. El paciente queda disponible para que un administrativo le asigne turnos, o para que él mismo complete su registro de usuario (ver Requerimiento 2).

2. El sistema debe permitir a un paciente ya dado de alta registrarse para obtener acceso (usuario y contraseña).

Entrada: DNI, nombre de usuario, contraseña.

Proceso: El sistema busca en la tabla pacientes un registro con ese DNI. Si no existe, solicita completar primero el alta de paciente (ver Requerimiento 1). Si existe y su usuarioID está en nulo, genera un id_usuario, calcula el hash de la contraseña, y guarda el registro en la tabla usuarios con rol "paciente", vinculándolo al paciente mediante usuarioID. Si el paciente ya tenía un usuario asociado, rechaza la operación.

Salida: Mensaje de operación exitosa y habilitación para iniciar sesión.

3. El sistema debe permitir a cualquier usuario (paciente, médico, secretario o administrador) iniciar sesión.

Entrada: nombre de usuario, contraseña.

Proceso: El sistema busca en la tabla usuarios un registro con ese nombre de usuario. Si no existe, o si el hash de la contraseña ingresada no coincide con contrasena_hash, rechaza el acceso. Si coincide, genera un token JWT que incluye el id_usuario y el rol, y lo devuelve.

Salida: Token JWT y redirección a la pantalla principal correspondiente al rol del usuario.

4. El sistema debe permitir al Administrador dar de alta un médico o un secretario, con su usuario de acceso.

Entrada: datos personales (nombre, apellido, fecha de nacimiento, domicilio, teléfono, DNI), especialidad (si es médico), nombre de usuario y contraseña inicial.

Proceso: El sistema verifica que el DNI no esté ya registrado en especialistas o secretarios según corresponda, inserta el registro, y genera su usuario asociado en la tabla usuarios con el rol correspondiente, vinculado mediante la columna de enlace (usuarioID, a agregar en especialistas — ver nota en "Modelo de datos"; usuario en secretarios).

Salida: Mensaje de alta exitosa; el nuevo médico o secretario ya puede iniciar sesión.

Grupo 2 — Turnos en línea

5. El sistema debe permitir consultar las especialidades y los médicos disponibles.

Entrada: (ninguna, o un filtro opcional por especialidad).

Proceso: El sistema consulta la tabla especialidades y, si se indicó un filtro, la tabla especialistas filtrando por el id_especialidad correspondiente.

Salida: Lista de especialidades y/o médicos, con nombre, apellido y especialidad de cada uno.

6. El sistema debe permitir consultar los horarios disponibles de un médico en una fecha determinada.

Entrada: id_especialista, fecha.

Proceso: El sistema consulta la tabla turnos filtrando por ese especialista y esa fecha, y calcula qué franjas horarias de su agenda configurada (Requerimiento 15) todavía no tienen un turno en estado "activo".

Salida: Lista de horarios libres para ese médico en esa fecha.

7. El sistema debe permitir a un paciente solicitar un turno estando conectado a internet.

Entrada: id_paciente, id_especialista, fecha, hora.

Proceso: El sistema valida, dentro de una transacción, que ese horario para ese especialista siga libre. Si está libre, inserta el registro en la tabla turnos con estado "activo". Si ya fue tomado por otro paciente, rechaza la operación.

Salida: Mensaje de turno confirmado, o aviso de que el horario ya no está disponible.

8. El sistema debe permitir a un paciente cancelar un turno propio.

Entrada: id_turno.

Proceso: El sistema verifica que el turno pertenezca al paciente que solicita la cancelación y que su estado sea "activo". Actualiza el campo estado a "cancelado" y libera el horario para que otros pacientes puedan solicitarlo.

Salida: Mensaje de cancelación exitosa; se dispara una notificación al médico/secretario (ver Requerimiento 14).

Grupo 3 — Turnos offline, sincronización e historial

9. El sistema debe permitir a un paciente solicitar un turno sin conexión a internet.

Entrada: id_paciente, id_especialista, fecha, hora.

Proceso: La app detecta que no hay conexión, valida localmente que el horario no esté ya tomado entre los turnos ya sincronizados, y guarda el turno en SQLite con estado "pendiente de sincronizar", agregándolo a la cola de operaciones pendientes.

Salida: Mensaje de "turno guardado, se confirmará al recuperar conexión".

10. El sistema debe sincronizar automáticamente los turnos pendientes cuando el dispositivo recupera conexión.

Entrada: (ninguna; se dispara automáticamente al detectar conectividad).

Proceso: El SyncManager recorre la cola de operaciones pendientes guardadas en SQLite y las envía una por una a la API. Por cada turno, el servidor valida en una transacción que el horario siga libre; si lo está, lo confirma (estado "activo" en turnos); si ya fue tomado, lo marca como rechazado.

Salida: Para cada turno sincronizado, mensaje de "turno confirmado" o de "el horario ya no está disponible, elegí otro".

11. El sistema debe permitir a un paciente consultar su historial de turnos (pasados y próximos).

Entrada: id_paciente.

Proceso: El sistema consulta la tabla turnos filtrando por ese paciente, ordenados por fecha, e incluye su estado (activo, cancelado, reprogramado).

Salida: Lista de turnos del paciente con fecha, hora, médico y estado.

12. El sistema debe permitir a un médico o secretario consultar el historial clínico de un paciente.

Entrada: id_paciente (o DNI).

Proceso: El sistema consulta la tabla historialclinico filtrando por pacienteDNI, solo si quien consulta tiene rol "especialista" o "secretario" (validado por el JWT).

Salida: Lista de consultas anteriores del paciente, con fecha, diagnóstico y tratamiento.

Grupo 4 — Perfil, notificaciones y gestión

13. El sistema debe permitir a un paciente consultar y editar sus propios datos de filiación.

Entrada: id_paciente, y los campos a modificar (domicilio, teléfono, obra social).

Proceso: El sistema verifica que el usuario autenticado sea el dueño de ese id_paciente (o un administrador), y actualiza los campos correspondientes en la tabla pacientes.

Salida: Mensaje de datos actualizados correctamente.

14. El sistema debe enviar recordatorios automáticos de turnos próximos.

Entrada: (ninguna; se ejecuta en forma programada, por ejemplo 24hs antes del turno).

Proceso: Una tarea programada (Azure Functions) consulta la tabla turnos buscando los que están en estado "activo" y cuya fecha/hora cae dentro de la ventana de aviso. Por cada uno, arma el mensaje y lo envía por push (FCM) y/o WhatsApp, y guarda el registro en la tabla notificaciones vinculado al usuario y al turno correspondientes.

Salida: Notificación recibida por el paciente; registro guardado para evitar duplicados.

15. El sistema debe permitir a un médico o secretario configurar la disponibilidad de agenda de un especialista.

Entrada: id_especialista, días y horarios de atención, duración de cada consulta.

Proceso: El sistema valida que quien realiza la carga tenga rol "especialista" (sobre su propia agenda) o "secretario", y guarda la configuración que luego se usa para calcular los horarios libres (Requerimiento 6).

Salida: Mensaje de agenda actualizada correctamente.

Lógica del lado de la app (Flutter)

Siguiendo las capas de Clean Architecture ya definidas, esto es lo que hay que codear en cada una:

Domain (entidades y reglas, sin depender de Flutter ni de la base)

- Entidades: `Usuario`, `Paciente`, `Especialista`, `Turno`
- Use Cases: `IniciarSesionUseCase`, `SolicitarTurnoUseCase`, `CancelarTurnoUseCase`, `ConsultarHistorialUseCase`, `SincronizarTurnosUseCase`

Data (acceso a datos)

- `AuthRepository`: login contra la API, guarda el JWT en `flutter_secure_storage`
- `TurnoRepository`: decide si lee/escribe en SQLite o en la API según haya conexión
- Data sources: cliente HTTP (consumiendo los endpoints de la sección anterior) + helper de SQLite (tablas locales: turnos, cola de sincronización)
- `SyncManager`: recorre la cola de operaciones pendientes y las reenvía a `/turnos/sync` cuando detecta conexión

Presentation (pantallas + estado)

- Un ViewModel por pantalla de la sección anterior (ej. `AgendaViewModel`, `HistorialViewModel`)
- Detección de conectividad (paquete `connectivity_plus` o similar) para decidir el flujo online/offline en tiempo real

Esta separación es la que permite repartir el trabajo sin pisarse: quien hace una pantalla solo necesita saber qué Use Case llamar y qué datos recibe de vuelta, sin tocar cómo se guardan o sincronizan los datos.

División de tareas sugerida

Ahora separado por número de requerimiento (sección anterior), para que cada uno arme de punta a punta su pantalla + su lógica de backend sin depender de que otro termine primero.

Persona	A cargo de	Requerimientos
Persona 1	Cuentas y acceso	1 a 4
Persona 2	Turnos en línea	5 a 8
Persona 3	Turnos offline, sincronización e historial	9 a 12
Persona 4	Perfil, notificaciones y gestión de agenda	13 a 15

Importante: los Requerimientos 1 a 4 (Persona 1) definen el login con JWT del que dependen todos los demás. Conviene que esa parte quede lista primero, o que mientras tanto el resto trabaje con un token de prueba hardcodeado, para no bloquearse entre sí.

Checklist del prototipo funcional (Parte V)

Lo que pide la consigna, para ir tildando a medida que se completa:

- [ ] Listar turnos disponibles de un médico
- [ ] Registrar (solicitar) un turno
- [ ] Guardar datos localmente (SQLite)
- [ ] Simular modo sin conexión
- [ ] Recuperar información al reabrir la app
- [ ] Mostrar mensaje de sincronización
- [ ] Captura: pantalla principal
- [ ] Captura: alta de turno
- [ ] Captura: funcionamiento offline
- [ ] Captura: sincronización de datos
- [ ] Código fuente empaquetado (ZIP)
- [ ] APK o link al repositorio de GitHub
- [ ] Manual breve de uso